

Compiler Design

Lab Assessment 2

Student Name: Avyakth Shriram

Register Number: 24BCE5461

PROGRAM 1

first_and_follow.c

Aim:

To write a C program that parses Context-Free Grammar (CFG) productions and dynamically computes the FIRST and FOLLOW sets for all non-terminals, aiding top-down parser implementation.

Objective:

- ✓ To parse, tokenize, and store grammar production rules, non-terminals, and terminals.
- ✓ To recursively/iteratively evaluate FIRST() sets by processing terminal prefixes and handling nullability (epsilon ϵ).
- ✓ To evaluate FOLLOW() sets by scanning production right-hand sides, accounting for adjacent grammar symbols and end-of-input markers (\$).

C Program Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_PROD 20
#define MAX_SYMBOLS 50
#define MAX_LEN 20
```

```

typedef struct {
    char lhs[MAX_LEN];
    char rhs[MAX_PROD][MAX_LEN];
    int rhs_count;
} Production;

Production prods[MAX_PROD];
int prod_count = 0;

char non_terminals[MAX_SYMBOLS][MAX_LEN];
int nt_count = 0;

char terminals[MAX_SYMBOLS][MAX_LEN];
int t_count = 0;

int first_set[MAX_SYMBOLS][MAX_SYMBOLS];
int follow_set[MAX_SYMBOLS][MAX_SYMBOLS];

int get_nt_index(char *str) {
    for (int i = 0; i < nt_count; i++) {
        if (strcmp(non_terminals[i], str) == 0) return i;
    }
    return -1;
}

int get_t_index(char *str) {
    for (int i = 0; i < t_count; i++) {
        if (strcmp(terminals[i], str) == 0) return i;
    }
    return -1;
}

void add_nt(char *str) {
    if (get_nt_index(str) == -1 && strlen(str) > 0) {
        strcpy(non_terminals[nt_count++], str);
    }
}

void add_t(char *str) {
    if (get_nt_index(str) == -1 && get_t_index(str) == -1 && strlen(str) > 0) {
        strcpy(terminals[t_count++], str);
    }
}

```

```

int is_nt(char *str) {
    return get_nt_index(str) != -1;
}

int tokenize_rhs(char *rhs_raw, char tokens[MAX_PROD][MAX_LEN]) {
    int t_idx = 0;
    int i = 0;
    int len = strlen(rhs_raw);

    while (i < len) {
        if (rhs_raw[i] == 'i' && i + 1 < len && rhs_raw[i+1] == 'd') {
            strcpy(tokens[t_idx++], "id");
            i += 2;
        } else if (i + 1 < len && rhs_raw[i+1] == '\n') {
            tokens[t_idx][0] = rhs_raw[i];
            tokens[t_idx][1] = '\n';
            tokens[t_idx][2] = '\0';
            t_idx++;
            i += 2;
        } else {
            tokens[t_idx][0] = rhs_raw[i];
            tokens[t_idx][1] = '\0';
            t_idx++;
            i++;
        }
    }
    return t_idx;
}

void compute_first() {
    int change = 1;
    while (change) {
        change = 0;
        for (int i = 0; i < prod_count; i++) {
            int lhs_idx = get_nt_index(prods[i].lhs);

            for (int j = 0; j < prods[i].rhs_count; j++) {
                char tokens[MAX_PROD][MAX_LEN];
                int token_count = tokenize_rhs(prods[i].rhs[j], tokens);

                if (token_count == 1 && strcmp(tokens[0], "e") == 0) {
                    int eps_idx = get_t_index("e");

```

```

        if (!first_set[lhs_idx][eps_idx]) {
            first_set[lhs_idx][eps_idx] = 1;
            change = 1;
        }
        continue;
    }

    int k = 0;
    int continue_loop = 1;
    while (k < token_count && continue_loop) {
        continue_loop = 0;
        char *sym = tokens[k];

        if (!is_nt(sym)) {
            int t_idx = get_t_index(sym);
            if (!first_set[lhs_idx][t_idx]) {
                first_set[lhs_idx][t_idx] = 1;
                change = 1;
            }
        } else {
            int sym_nt_idx = get_nt_index(sym);
            for (int t = 0; t < t_count; t++) {
                if (strcmp(terminals[t], "e") != 0 && first_set[sym_nt_idx][t]) {
                    if (!first_set[lhs_idx][t]) {
                        first_set[lhs_idx][t] = 1;
                        change = 1;
                    }
                }
            }
            if (first_set[sym_nt_idx][get_t_index("e")]) {
                continue_loop = 1;
            }
        }
        k++;
    }

    if (k == token_count && continue_loop) {
        int eps_idx = get_t_index("e");
        if (!first_set[lhs_idx][eps_idx]) {
            first_set[lhs_idx][eps_idx] = 1;
            change = 1;
        }
    }
}

```

```

    }
}
}

```

```

void compute_follow() {
    follow_set[get_nt_index(prods[0].lhs)][get_t_index("$")] = 1;

    int change = 1;
    while (change) {
        change = 0;
        for (int i = 0; i < prod_count; i++) {
            int lhs_idx = get_nt_index(prods[i].lhs);

            for (int j = 0; j < prods[i].rhs_count; j++) {
                char tokens[MAX_PROD][MAX_LEN];
                int token_count = tokenize_rhs(prods[i].rhs[j], tokens);

                for (int k = 0; k < token_count; k++) {
                    char *sym = tokens[k];
                    if (!is_nt(sym)) continue;
                    int sym_nt_idx = get_nt_index(sym);

                    int next_pos = k + 1;
                    int continue_loop = 1;

                    while (next_pos < token_count && continue_loop) {
                        continue_loop = 0;
                        char *next_sym = tokens[next_pos];

                        if (!is_nt(next_sym)) {
                            int t_idx = get_t_index(next_sym);
                            if (!follow_set[sym_nt_idx][t_idx]) {
                                follow_set[sym_nt_idx][t_idx] = 1;
                                change = 1;
                            }
                        }
                        else {
                            int next_nt_idx = get_nt_index(next_sym);
                            for (int t = 0; t < t_count; t++) {
                                if (strcmp(terminals[t], "e") != 0 && first_set[next_nt_idx][t]) {
                                    if (!follow_set[sym_nt_idx][t]) {
                                        follow_set[sym_nt_idx][t] = 1;
                                        change = 1;
                                    }
                                }
                            }
                        }
                        next_pos++;
                    }
                }
            }
        }
    }
}

```



```

// Create a temporary sorted array of matching items for the current row
char matches[MAX_SYMBOLS][MAX_LEN];
int m_count = 0;

for (int t = 0; t < t_count; t++) {
    if (set_matrix[nt_idx][t]) {
        strcpy(matches[m_count++], terminals[t]);
    }
}

// Apply order sort
qsort(matches, m_count, MAX_LEN, compare_terminals);

for (int i = 0; i < m_count; i++) {
    printf("%s", matches[i]);
    if (i < m_count - 1) printf(",");
}
printf("{}");
}

int main() {
    int n;
    printf("Enter the no. of production:");
    if (scanf("%d", &n) != 1) return 1;
    prod_count = n;

    char raw_line[MAX_LEN * 3];
    for (int i = 0; i < n; i++) {
        scanf("%s", raw_line);

        char *arrow = strstr(raw_line, "->");
        *arrow = '\0';
        strcpy(prods[i].lhs, raw_line);
        add_nt(prods[i].lhs);

        char *rhs_part = arrow + 2;
        char *token = strtok(rhs_part, "/");
        prods[i].rhs_count = 0;
        while (token != NULL) {
            strcpy(prods[i].rhs[prods[i].rhs_count++], token);
            token = strtok(NULL, "/");
        }
    }
}

```

```

}

// Dynamic extraction phase for terminal matching sets
for (int i = 0; i < prod_count; i++) {
    for (int j = 0; j < prods[i].rhs_count; j++) {
        char tokens[MAX_PROD][MAX_LEN];
        int token_count = tokenize_rhs(prods[i].rhs[j], tokens);
        for (int k = 0; k < token_count; k++) {
            if (strcmp(tokens[k], "ε") != 0) {
                add_t(tokens[k]);
            }
        }
    }
}
add_t("ε");
add_t("$");

compute_first();
compute_follow();

printf("\nFirst\n");
for (int i = 0; i < nt_count; i++) {
    printf("First(%s)=", non_terminals[i]);
    print_set(first_set, i);
    printf("\n");
}

printf("\nFollow\n");
for (int i = 0; i < nt_count; i++) {
    printf("Follow(%s)=", non_terminals[i]);
    print_set(follow_set, i);
    printf("\n");
}

return 0;
}

```

Output/Screenshots:

```
scope1@AB3-302-Sys56: ~/Av X
scope1@AB3-302-Sys56:~/AvyakthS/Compiler-Design-Lab$ tree
.
├── Lab_Assignment_1
│   ├── Compiler Design Lab - Assignment 1.docx
│   ├── Compiler Design Lab - Assignment 1.pdf
│   ├── input.c
│   ├── lex.yy.c
│   ├── lexer.l
│   ├── symbol-table-generator_plus_lexer
│   ├── symbol_table_generator
│   └── symbol_table_generator.c
└── Lab_Assignment_2
    ├── calc
    ├── calc.l
    ├── first_and_follow
    ├── first_and_follow.c
    ├── lex.yy.c
    ├── sort
    └── sort.l

3 directories, 15 files
scope1@AB3-302-Sys56:~/AvyakthS/Compiler-Design-Lab$ cd Lab_Assignment_2
scope1@AB3-302-Sys56:~/AvyakthS/Compiler-Design-Lab/Lab_Assignment_2$ ./first_and_follow
Enter the no. of production:5
E->TE'
E'->+TE'/e
T->FT'
T'->*FT'/e
F->(E)/id

First
First(E)={(,id}
First(E')={+,e}
First(T)={(,id}
First(T')={*,e}
First(F)={(,id}

Follow
Follow(E)={),}$}
Follow(E')={),}$}
Follow(T)={),+,$}
Follow(T')={),+,$}
Follow(F)={),*,+,$}
scope1@AB3-302-Sys56:~/AvyakthS/Compiler-Design-Lab/Lab_Assignment_2$ |
```

PROGRAM 2

calc.1

Aim:

To design and implement a simple interactive arithmetic calculator using Lex (Flex) to perform token recognition and state-driven calculation for two-number arithmetic operations.

Objective:

- ✓ To construct regular expressions in Lex that identify numbers (integers and floats) and basic arithmetic operators (+, -, *, /, %).
- ✓ To implement a finite state tracker that validates the input sequence format ([Number] [Operator] [Number]).
- ✓ To execute calculations and gracefully handle invalid syntax or division-by-zero errors.

C Program Code:

```
%{  
#include <stdio.h>  
#include <stdlib.h>  
  
double num1 = 0, num2 = 0;  
char op = 0;  
  
/* State tracker:  
0 = expect first number  
1 = expect operator
```

```

    2 = expect second number
    3 = calculation ready
*/
int state = 0;

/* Helper function to perform the math */
void calculate() {
    if (state != 3) {
        printf("Error: Incomplete expression. Please enter format: [Number] [Operator]
[Number]\n");
        return;
    }

    switch(op) {
        case '+': printf("Result: %g\n", num1 + num2); break;
        case '-': printf("Result: %g\n", num1 - num2); break;
        case '*': printf("Result: %g\n", num1 * num2); break;
        case '/':
            if (num2 != 0.0) {
                printf("Result: %g\n", num1 / num2);
            } else {
                printf("Error: Division by zero\n");
            }
            break;
        case '%':
            printf("Result: %d\n", (int)num1 % (int)num2);
            break;
    }
}

/* Helper function to reset for the next calculation */
void reset() {
    num1 = 0;
    num2 = 0;
    op = 0;
    state = 0;
    printf("Enter calculation (e.g., 5 + 3): ");
}
}%}

%%

[0-9]+(\\. [0-9]+)? {

```

```

if (state == 0) {
    num1 = atof(yytext);
    state = 1; /* Now look for an operator */
}
else if (state == 2) {
    num2 = atof(yytext);
    state = 3; /* Now ready to calculate */
}
else if (state == 3) {
    printf("Error: Only two numbers are allowed per calculation.\n");
}
else {
    printf("Error: Expected an operator, but got a number.\n");
}
}

[+\-*/%] {
    if (state == 1) {
        op = yytext[0];
        state = 2; /* Now look for the second number */
    }
    else {
        printf("Error: Operator '%s' is out of place.\n", yytext);
    }
}

\n {
    /* When the user hits Enter, calculate and reset */
    if (state > 0) {
        calculate();
    }
    reset();
}

[ \t] { /* Ignore spaces and tabs */ }

. { printf("Invalid character: %s\n", yytext); }

%%

int yywrap() {
    return 1;
}

```

```
int main() {  
    printf("--- Two-Number Lex Calculator ---\n");  
    printf("Format: [Number] [Operator] [Number]\n");  
    reset();  
    yylex();  
    return 0;  
}
```

Output/Screenshots:

```
scope1@AB3-302-Sys56: ~/Av X
scope1@AB3-302-Sys56:~/AvyakthS/Compiler-Design-Lab$ tree
.
├── Lab_Assignment_1
│   ├── Compiler Design Lab - Assignment 1.docx
│   ├── Compiler Design Lab - Assignment 1.pdf
│   ├── input.c
│   ├── lex.yy.c
│   ├── lexer.l
│   ├── symbol-table-generator_plus_lexer
│   ├── symbol_table_generator
│   └── symbol_table_generator.c
└── Lab_Assignment_2
    ├── calc
    ├── calc.l
    ├── first_and_follow
    ├── first_and_follow.c
    ├── lex.yy.c
    ├── sort
    └── sort.l

3 directories, 15 files
scope1@AB3-302-Sys56:~/AvyakthS/Compiler-Design-Lab$ cd Lab_Assignment_2
scope1@AB3-302-Sys56:~/AvyakthS/Compiler-Design-Lab/Lab_Assignment_2$ ./calc
--- Two-Number Lex Calculator ---
Format: [Number] [Operator] [Number]
Enter calculation (e.g., 5 + 3): 67 * 69
Result: 4623
Enter calculation (e.g., 5 + 3): 0 * 9
Result: 0
Enter calculation (e.g., 5 + 3): 11 * 99
Result: 1089
Enter calculation (e.g., 5 + 3): 1000000 * 9999999
Result: 1e+13
Enter calculation (e.g., 5 + 3): 67 * 68 * 69
Error: Operator '*' is out of place.
Error: Only two numbers are allowed per calculation.
Result: 4556
Enter calculation (e.g., 5 + 3): 2 * 3 * 4
Error: Operator '*' is out of place.
Error: Only two numbers are allowed per calculation.
Result: 6
Enter calculation (e.g., 5 + 3): ^C
scope1@AB3-302-Sys56:~/AvyakthS/Compiler-Design-Lab/Lab_Assignment_2$ |
```

PROGRAM 3

sort.l

Aim:

To develop a Lex specification file that extracts numerical sequences from standard input and sorts them in ascending order using an embedded sorting algorithm.

Objective:

- ✓ To write Lex regular expressions that capture positive/negative integers and floating-point numbers while filtering out whitespace and invalid characters.
- ✓ To store tokenized numbers into an internal dynamic/static array buffer.
- ✓ To apply an inline sorting routine (Bubble Sort) when a newline delimiter (\n) is encountered and print the sorted output.

C Program Code:

```
%{  
#include <stdio.h>  
#include <stdlib.h>  
  
#define MAX_SIZE 100  
  
double arr[MAX_SIZE];  
int count = 0;  
  
/* Standard Bubble Sort implementation in C */
```

```

void bubble_sort() {
    if (count == 0) {
        return; /* Nothing to sort */
    }

    for (int i = 0; i < count - 1; i++) {
        for (int j = 0; j < count - i - 1; j++) {
            if (arr[j] > arr[j+1]) {
                /* Swap the elements */
                double temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }

    /* Print the sorted array */
    printf("Sorted result: ");
    for (int i = 0; i < count; i++) {
        printf("%g ", arr[i]);
    }
    printf("\n");
}

/* Reset the array counter for the next input */
void reset() {
    count = 0;
    printf("Enter numbers separated by spaces: ");
}

%%

-?[0-9]+(\.[0-9]+)? {
    /* When a number is found, add it to the array */
    if (count < MAX_SIZE) {
        arr[count] = atof(yytext);
        count++;
    } else {
        printf("Warning: Array full (max %d). Ignoring %s\n", MAX_SIZE, yytext);
    }
}

```

```
\n {  
    /* When Enter is pressed, sort the array and reset */  
    bubble_sort();  
    reset();  
}  
  
[ \\t] { /* Ignore spaces and tabs */ }  
  
. {  
    /* Catch letters or special characters */  
    printf("Invalid character ignored: %s\\n", yytext);  
}  
  
%%  
  
int yywrap() {  
    return 1;  
}  
  
int main() {  
    printf("--- Lex Bubble Sorter ---\\n");  
    reset();  
    yylex();  
    return 0;  
}
```

Output/Screenshots:

```
scope1@AB3-302-Sys56: ~/Av \ X + v
scope1@AB3-302-Sys56:~/AvyakthS/Compiler-Design-Lab$ tree
.
├── Lab_Assignment_1
│   ├── Compiler Design Lab - Assignment 1.docx
│   ├── Compiler Design Lab - Assignment 1.pdf
│   ├── input.c
│   ├── lex.yy.c
│   ├── lexer.l
│   ├── symbol-table-generator_plus_lexer
│   ├── symbol_table_generator
│   └── symbol_table_generator.c
└── Lab_Assignment_2
    ├── calc
    ├── calc.l
    ├── first_and_follow
    ├── first_and_follow.c
    ├── lex.yy.c
    ├── sort
    └── sort.l

3 directories, 15 files
scope1@AB3-302-Sys56:~/AvyakthS/Compiler-Design-Lab$ cd Lab_Assignment_2
scope1@AB3-302-Sys56:~/AvyakthS/Compiler-Design-Lab/Lab_Assignment_2$ ./sort
--- Lex Bubble Sorter ---
Enter numbers separated by spaces: 11 22 33 44 55 66 77 88 99 90 81 72 63 54 45 36 27 18 09
Sorted result: 9 11 18 22 27 33 36 44 45 54 55 63 66 72 77 81 88 90 99
Enter numbers separated by spaces: 555 500 666 600 777 700
Sorted result: 500 555 600 666 700 777
Enter numbers separated by spaces: -47 74 -343 343 -666 999
Sorted result: -666 -343 -47 74 343 999
Enter numbers separated by spaces: 5 4 3 2 1 0 -1 -2 -3 -4 -5
Sorted result: -5 -4 -3 -2 -1 0 1 2 3 4 5
Enter numbers separated by spaces: ^C
scope1@AB3-302-Sys56:~/AvyakthS/Compiler-Design-Lab/Lab_Assignment_2$ |
```

THANK YOU